EnvShare — Free-Tier Technology Stack

Version: 2.0
Date: October 24, 2026
Based on: `EnvShare_PRD.md`
Scope: MVP implementation using free or open-source tools wherever possible
Cost target: $0/month for development, demo, coursework, portfolio, and small-scale personal use

1. Purpose

This document recommends a **free-tier-first technology stack** for EnvShare, a secure web application for teams to manage, share, and audit project environment variables.

The stack is designed to satisfy the PRD's MVP requirements:

- User registration and authentication
- Projects and team membership
- Multiple environments
- Standard and sensitive variables
- Encryption and masked display of sensitive values
- Bulk `.env` import with review
- Change history and auditability
- Server-side authorization

This version prioritizes tools that are either **open source** or available on a **permanent free tier**, while avoiding paid enterprise services during the MVP phase.

2. Recommended Free-Tier Stack

Layer	Recommended Technology	Cost	Why It Is Recommended
Frontend	Next.js 15+ App Router, React, TypeScript	Free / open source	Full-stack React framework with protected routes, server rendering, and strong developer experience
UI	Tailwind CSS + shadcn/ui	Free / open source	Consistent styling and accessible dashboard components
Backend	Next.js Route Handlers / Server Actions	Free / open source	Handles CRUD, authorization, imports, and audit history without a separate paid backend
Validation	Zod	Free / open source	Strong validation for authentication, variables, and `.env` imports

Layer	Recommended Technology	Cost	Why It Is Recommended
Authentication	Auth.js v5	Free / open source	Supports credentials, sessions, password hashing, and future OAuth/SSO
Database	PostgreSQL	Free / open source	Relational integrity, transactions, constraints, and audit-friendly modeling
Database hosting	Neon Free Plan	$0	Managed serverless Postgres with a permanent free tier
ORM	Prisma	Free / open source	Type-safe queries, migrations, and transactional writes
Secret encryption	Node.js `crypto` with AES-256-GCM	Free / built in	Encrypts sensitive values at rest without a paid KMS
Key storage	Environment variables / Vercel encrypted env vars	Free	Suitable for MVP; upgrade to KMS/Vault later
Deployment	Vercel Hobby	$0	Native Next.js hosting, Git deployments, and preview environments
CI/CD	GitHub Actions	Free for public repos / generous free quota	Automated tests, linting, and deployment checks
Error monitoring	Sentry Free Developer Plan	$0	Captures production errors without paid observability tooling
Testing	Vitest + Playwright	Free / open source	Unit, integration, and end-to-end testing

3. Important Free-Tier Constraints

This stack can run at **$0/month**, but it is best suited for:

- Course projects
- Hackathons
- Portfolios
- Personal projects
- Demos
- Small-scale internal prototypes

It is **not fully free for a commercial production product** because:

- Vercel's Hobby plan is intended for personal, non-commercial use.
- Neon's free tier has storage, compute, branch, and project limits.
- Sentry's free plan has usage and retention limits.
- A custom domain is not free.
- Managed KMS or Vault may become billable at scale.

For a real commercial deployment, plan to move to Vercel Pro or another commercial hosting provider, and consider managed KMS, paid monitoring, and a production database plan.

4. Frontend Recommendation

4.1 Next.js + React + TypeScript

Recommendation: Use **Next.js 15+ with the App Router**, **React**, and **TypeScript**.

Next.js is free, open source, and ideal for EnvShare because the product is an authenticated dashboard application. It needs project lists, environment views, variable tables, `.env` import previews, and audit-history timelines.

Benefits:

- Server Components reduce the amount of sensitive data sent to the browser.
- Protected routes can be checked on the server before rendering.
- Server Actions simplify secure form submissions and mutations.
- TypeScript improves safety across users, projects, environments, and variables.
- The frontend, backend, validation, database access, and tests can live in one free GitHub repository.

4.2 Tailwind CSS + shadcn/ui

Recommendation: Use **Tailwind CSS** with **shadcn/ui**.

Both are free and open source. They satisfy the PRD's requirement for a consistent UI system and clear validation/error messaging.

Recommended components:

- Data tables for variables and history
- Forms for projects, environments, and variables
- Dialogs for confirmations and secret reveals
- Tabs for environment switching
- Badges for `Standard` and `Sensitive`
- Diff-style preview for `.env` imports
- Toasts for success, warning, and error feedback

5. Backend Recommendation

5.1 Next.js Server Layer

Recommendation: Use **Next.js Route Handlers and Server Actions** instead of a separate Express, NestJS, Django, or Spring Boot backend.

This avoids additional hosting costs and operational complexity. EnvShare's MVP backend mainly performs:

- Authentication and session validation
- Project membership authorization
- CRUD operations
- `.env` parsing
- Encryption and decryption
- Audit-history creation
- Transactional database writes

These workloads fit well within Next.js's server layer.

5.2 Recommended Project Structure

```
src/
  app/
    (auth)/
      login/
      register/
    dashboard/
      projects/
      settings/
    api/
      auth/
      projects/
      variables/
      imports/
  lib/
    auth.ts
    authorization.ts
    crypto.ts
    env-parser.ts
    audit.ts
    validation.ts
  server/
    services/
      project-service.ts
      environment-service.ts
      variable-service.ts
      import-service.ts
      history-service.ts
  db/
    schema.prisma
    migrations/
```

5.3 Validation with Zod

Recommendation: Use **Zod** for all external input.

The PRD identifies important `.env` parsing edge cases:

- Blank lines
- Comments
- Duplicate keys
- Values containing `=`
- Quoted values
- Multiline values
- Empty values
- Invalid lines
- Oversized imports

Validate all input before it reaches business logic or the database.

6. Authentication Recommendation

6.1 Auth.js v5

Recommendation: Use **Auth.js v5**.

Auth.js is free and open source. It supports:

- Email/password registration
- Secure password hashing
- Login sessions
- HTTP-only cookies
- Protected routes
- Optional GitHub or Google OAuth
- Future enterprise SSO expansion

6.2 MVP Authentication Design

For the free-tier MVP, support:

- Email and password registration
- Email and password login
- Secure password hashing
- Server-side session validation

- HTTP-only secure cookies
- Optional GitHub OAuth login

Do not store plaintext passwords. Use a modern password-hashing algorithm such as **bcrypt** or **argon2id**.

6.3 Authorization Model

The MVP should implement:

Role	Permissions
Project creator	Create project, add members, manage environments and variables
Project member	View and edit authorized project variables
Non-member	No access to project data or secrets

Every sensitive operation must be authorized on the server:

- Viewing a project
- Viewing an environment
- Reading a variable
- Creating, updating, or deleting a variable
- Revealing a sensitive value
- Importing `.env` content
- Viewing change history

7. Database Recommendation

7.1 PostgreSQL

Recommendation: Use **PostgreSQL**.

PostgreSQL is free and open source. It is the best database choice for EnvShare because the application has strongly relational data:

- Users belong to projects through memberships
- Projects contain environments
- Environments contain variables
- Variables have versions and audit records
- Variable changes must be recorded consistently

PostgreSQL provides:

- ACID transactions
- Foreign keys

- Unique constraints
- JSONB for audit metadata
- Indexing
- Strong data integrity

7.2 Neon Free Plan

Recommendation: Use **Neon's Free Plan** for hosted PostgreSQL.

Neon's free plan is suitable for an MVP because it includes:

- 100 projects
- 100 compute-hours per project per month
- 1 GB storage per project
- Up to 10 branches per project
- Scale-to-zero behavior after inactivity

This makes it a strong $0 option for development, demos, and low-traffic usage.

7.3 Prisma ORM

Recommendation: Use **Prisma**.

Prisma is free and open source. It provides:

- A declarative schema
- Type-safe database queries
- Migrations
- Relation modeling
- Transaction support
- Easy integration with Next.js and TypeScript

8. Data Model

8.1 Proposed Schema

```
User
  id
  email
  name
  password_hash
  created_at
  updated_at

Project
```

```
  id
  name
  description
  created_by_user_id
  created_at
  updated_at

ProjectMember
  id
  project_id
  user_id
  role
  created_at

Environment
  id
  project_id
  name
  created_at
  updated_at

Variable
  id
  environment_id
  key
  value_ciphertext
  value_hash
  classification
  encryption_key_version
  created_by_user_id
  created_at
  updated_at

VariableVersion
  id
  variable_id
  actor_id
  action
  old_value_hash
  new_value_hash
  metadata
  created_at

AuditEvent
  id
  project_id
  actor_id
  action
  resource_type
  resource_id
  metadata
  created_at
```

8.2 Transactional History

Variable updates, deletions, and imports should write both the new state and audit records in one transaction.

```
BEGIN TRANSACTION
  update variable
  insert variable_version
  insert audit_event
COMMIT
```

If the history insert fails, roll back the variable update. This prevents configuration changes from existing without an audit trail.

9. Secret Protection on a Free Tier

9.1 Encryption Approach

Recommendation: Use **AES-256-GCM encryption** implemented with Node.js's built-in `crypto` module.

This avoids the need for a paid KMS during the MVP.

Recommended process:

1. Generate a strong random master encryption key.
2. Store it only in a secure environment variable.
3. Generate a unique initialization vector for each encrypted value.
4. Encrypt sensitive values using AES-256-GCM.
5. Store ciphertext, IV, authentication tag, and key version.
6. Decrypt only on the server during an authorized reveal.
7. Audit every reveal event.

9.2 MVP Key Management

For the free-tier MVP:

- Generate the master key locally using a secure random generator.
- Store it in Vercel environment variables or a local `.env.local` file.
- Never commit it to GitHub.
- Use different keys for development and production.
- Rotate the key manually if needed.

For production at scale, migrate to:

- AWS KMS
- Google Cloud KMS
- HashiCorp Vault

The application should store an `encryption_key_version` field so keys can be rotated later without breaking existing encrypted values.

9.3 UI Masking

Sensitive values should be masked by default:

```
DATABASE_URL = ••••••••••••••••••
```

A reveal action should:

- Require an authenticated session
- Verify project membership
- Verify permission to reveal secrets
- Create an audit event
- Avoid browser storage, logs, analytics, and error reports

9.4 Import Preview Safety

The `.env` import preview must not expose existing sensitive values.

Recommended behavior:

- Mask existing sensitive values
- Require explicit confirmation before saving incoming sensitive values
- Compare values using a secure hash or deterministic encryption
- Clearly identify new, changed, and unchanged keys
- Never log raw `.env` content

10. `.env` Import Recommendation

10.1 Parser

Recommendation: Build a server-side `.env` parser using TypeScript.

Do not send raw `.env` content to third-party APIs. Parse and validate it entirely inside the Next.js server.

The parser should:

- Ignore blank lines

- Ignore lines beginning with `#`
- Parse `KEY=VALUE` pairs
- Treat the first `=` as the separator
- Preserve additional `=` characters in values
- Detect duplicate keys
- Flag malformed lines
- Handle quoted values according to documented rules

10.2 Import Workflow

```
1. User pastes .env content
2. Server parses and validates content
3. Server compares parsed keys with existing variables
4. Server returns preview:
   - New keys
   - Changed keys
   - Unchanged keys
   - Invalid or conflicting entries
5. User reviews preview
6. User confirms import
7. Server applies changes transactionally
8. Server writes history and audit records
```

10.3 Import Rules

Case	Recommended MVP Behavior
Blank line	Ignore
Comment line	Ignore
Duplicate key in pasted content	Reject or require user resolution
Invalid line	Show validation error; do not silently save
Value contains `=`	Treat first `=` as separator
Empty value	Allow only if explicitly supported
Existing sensitive value	Mask in preview
Incoming sensitive value	Require explicit confirmation before saving
No changes	Do not create unnecessary history entries

11. Deployment Recommendation

11.1 Application Hosting: Vercel Hobby

Recommendation: Deploy the Next.js application to **Vercel's Hobby plan**.

Vercel Hobby is free and provides:

- Native Next.js support
- GitHub-based deployments
- Preview deployments
- HTTPS
- Serverless functions
- Global CDN
- Encrypted environment variables

Important limitation: Vercel Hobby is intended for **personal, non-commercial projects**. For a commercial product, upgrade to Vercel Pro or choose another hosting provider.

11.2 Database Hosting: Neon Free

Recommendation: Use **Neon Free** for PostgreSQL hosting.

Neon is a good free-tier choice because it provides:

- Managed Postgres
- Serverless compute
- Scale-to-zero
- Database branching
- Point-in-time recovery
- Easy Vercel integration

11.3 Recommended Environments

Environment	Purpose	Database
Local	Developer testing	Local Postgres or Neon development branch
Preview	Pull-request validation	Neon preview branch
Production	Live demo or MVP	Neon production database

For a zero-cost setup, avoid creating excessive branches and delete unused preview branches regularly.

11.4 CI/CD with GitHub Actions

Recommendation: Use **GitHub Actions**.

For public repositories, GitHub Actions is free within GitHub's usage limits. For private repositories, GitHub provides a monthly free Actions quota.

Recommended workflow:

1. Install dependencies
2. Run linting
3. Run TypeScript checks
4. Run unit tests
5. Run database migration checks
6. Deploy preview environment
7. Run end-to-end tests
8. Deploy to production after review

12. Monitoring and Testing

12.1 Error Monitoring

Recommendation: Use **Sentry's Free Developer Plan**.

Sentry helps identify production errors, but configure it carefully:

- Never send plaintext secrets to Sentry
- Scrub `.env` values from error messages
- Remove sensitive request bodies
- Avoid logging decrypted values
- Use project IDs and variable IDs instead of secret values

12.2 Testing Tools

Test Type	Tool	Cost	Coverage
Unit tests	Vitest	Free	`.env` parser, validation, crypto helpers
Integration tests	Vitest + test Postgres	Free	Auth, CRUD, encryption, history
End-to-end tests	Playwright	Free	Registration, projects, imports, reveals
Security tests	Custom tests + GitHub Dependabot	Free	Access control, injection, dependency risks

12.3 Priority Test Cases

- User can register and log in
- Invalid credentials are rejected
- Passwords are hashed
- Unauthenticated users cannot access projects
- Non-members cannot access project data
- Members can create and edit variables
- Sensitive values are encrypted in the database
- Sensitive values are masked by default
- Authorized users can reveal sensitive values
- Reveal events are audited
- `.env` parser ignores comments and blank lines
- Duplicate keys are handled safely
- Import preview identifies new, changed, and unchanged values
- Variable changes create history records
- History and variable updates are transactional

13. Security Requirements

13.1 Mandatory Controls

- Hash all passwords
- Encrypt sensitive variable values before database storage
- Mask sensitive values in the UI
- Authorize every project and secret operation on the server
- Use HTTPS
- Store session tokens in secure, HTTP-only cookies
- Apply CSRF protection to state-changing requests
- Rate-limit login, registration, reveal, and import endpoints
- Validate and sanitize all `.env` input
- Never log plaintext secrets
- Audit secret reveals and classification changes
- Store encryption keys only in environment variables or a secrets manager

13.2 Logging Rules

Never log:

- Plaintext sensitive values
- Raw `.env` import content
- Encryption keys
- Session tokens
- Password hashes

Log instead:

- User ID
- Project ID
- Environment ID
- Variable ID
- Action type
- Timestamp
- Request metadata
- Success or failure status

14. Future Upgrade Path

The free-tier stack should not block future growth. When EnvShare moves beyond a demo or personal MVP, upgrade gradually:

Future Need	Free-Tier Choice	Production Upgrade
Commercial hosting	Vercel Hobby	Vercel Pro, Render, Railway, Fly.io, or self-hosting
Database scale	Neon Free	Neon Launch/Scale or managed Postgres
Key management	Environment-variable master key	AWS KMS, Google Cloud KMS, or Vault
Monitoring	Sentry Free	Paid Sentry or alternative observability platform
SSO	Auth.js foundation	Enterprise identity-provider integration
Granular RBAC	Basic project roles	Viewer, editor, admin, auditor roles
Approvals	Not included in MVP	Workflow tables and approval states
Secret rotation	Encryption key versioning	Automated rotation and re-encryption
CLI access	Not included in MVP	API tokens and CLI client

15. Final Recommendation

For a **free-tier EnvShare MVP**, use:

```
Next.js + TypeScript
React + Tailwind CSS + shadcn/ui
Next.js Server Actions / Route Handlers
Zod
Auth.js v5
Prisma
PostgreSQL
Neon Free Plan
Node.js crypto with AES-256-GCM
Vercel Hobby
GitHub Actions
Sentry Free
Vitest + Playwright
```

This stack can be built and deployed at **$0/month** for coursework, demos, and personal use, while still meeting EnvShare's core security, encryption, authorization, import, and audit-history requirements.

For commercial production use, retain the same architecture but replace Vercel Hobby, Neon Free, and Sentry Free with paid plans or self-hosted alternatives once usage, compliance, and business requirements justify the cost.